The Quantum Search Bound behind HHL's Theorem 4(2)

imported from the AFP entry *Compressed Oracles*

Dustin Bryant

October 20, 2026

Abstract

This document is generated by Isabelle from the theory `HHL_Search_Bound` of the session `Vern_Paulsen_QC_Search`. It supplies the oracle lower bound used in the proof of Theorem 4(2) of Harrow, Hassidim and Lloyd: finding a zero of a random function with q quantum queries succeeds with probability at most $(5q+11)^2/N$. The bound is imported from the AFP entry *Compressed Oracles*; the session is kept apart from the main document because that entry builds on Complex_Bounded_Operators and Registers. Proofs are folded.

theory *HHL-Search-Bound*
imports *Compressed-Oracles.Find-Zero*
begin

1 The quantum search lower bound behind Theorem 4(2)

Theorem 4(2) of the HHL paper ends with a polynomial-query quantum algorithm for a search problem on $N = 2^n$ inputs, which is impossible. The bound is not reproved: it is *zero-finding-is-hard* of the AFP entry *Compressed-Oracles*, stated there for a random function h and any valid oracle program with q queries. We restate it as $sN \leq (5q+11)^2$ for a program finding a zero with average success s, the hypothesis of *search-speedup-impossible* in *HHL-Optimality*. The second part shows that exhaustive search, the computation the iteration speeds up, succeeds with probability at least $1/2$ when $|X| = |Y| = N$.

context *compressed-oracle* **begin**

theorem *search-query-bound*:
fixes *program* :: ‹('*mem*, '*x*, '*y*) *program*› **and** *adv-output* :: ‹'*x update* ⇒ '*mem update*› **and** *s* :: *real*
assumes ‹*valid-program program*› **and** ‹*norm initial-state* = *1*› **and** ‹*register adv-output*›
and *succ*: ‹*s* ≤ (∑ *h*∈*UNIV*. ∑ *x*|*h x* = *0*. *measurement-probability adv-output* (*exec-program h program initial-state*) *x*) / *CARD*('*x* ⇒ '*y*)›
shows ‹*s* * *N* ≤ (*5* * *real* (*query-count program*) + *11*)²›
⟨*proof*⟩

end

Exhaustive search finds a zero unless h has none: $(N-1)^{|X|}$ of the $N^{|X|}$ functions.

lemma *card-zero-free*: ‹*card* {*h* :: '*x*::*finite* ⇒ '*y*::{*finite*,*zero*}. ∀*x*. *h x* ≠ *0*} = (*CARD*('*y*) − *1*) ^ *CARD*('*x*)›
⟨*proof*⟩

lemma *no-zero-rare*:
fixes *N* :: *nat* **assumes** ‹*1* ≤ *N*›
shows ‹(*1* − *1* / *real N*) ^ *N* ≤ *1* / *2*›

⟨*proof*⟩

lemma *exhaustive-success*:
  assumes *N*: ‹*CARD*('*x*::*finite*) = *CARD*('*y*::{*finite*,*zero*})›
  shows ‹*1* / *2* ≤ *card* {*h* :: '*x* ⇒ '*y*. ∃ *x*. *h x* = *0*} / *CARD*('*x* ⇒ '*y*)›
⟨*proof*⟩

end